GUI-AC: Enhancing Continual Learning in GUI Agents

Can Lin
Beijing University of Posts and Telecommunications
lincan@bupt.edu.cn

Tao Feng
Tsinghua University
fengtao.hi@gmail.com

Hangjie Yuan
Zhejiang University
hj.yuan@zju.edu.cn

Dan Zhang
National University of Singapore
zhangdan25@nus.edu.sg

Yifan Zhu
Beijing University of Posts and Telecommunications
yifan_zhu@bupt.edu.cn

Zhonghong Ou
Beijing University of Posts and Telecommunications
zhonghong.ou@bupt.edu.cn

Abstract

Graphical User Interfaces (GUIs) serve as the dominant medium for human–computer interaction, yet building GUI agents that generalize across the vast diversity of real-world interface environments, with the same flexibility and robustness that humans naturally exhibit, remains unsolved. Notably, GUI data are inherently non-stationary: the continual emergence of previously unseen interface instances (e.g., novel domains and resolutions) induces persistent distribution shifts, significantly impeding the continual learning of existing GUI agents. Reinforcement fine-tuning (RFT) has attracted considerable attention as a promising approach. Nevertheless, RFT exhibits pronounced instability in its grounding capability, manifested as sharp reward discontinuities and high-variance oscillations. The imbalanced distribution of rollout outcomes introduces substantial noise into advantage estimation, leading to policy overconfidence. The fixed clipping bound suppresses the increase in policy probabilities needed to adapt to new distributions, leading to a collapse in exploration capacity. To address these challenges, we propose GUI-AC, a method that enhances the continual learning capability of GUI agents. GUI-AC introduces grounding certainty to support two core mechanisms: (i) Adaptive **A**dvantage, which down-weights noisy advantage estimates to prevent policy overconfidence; and (ii) Dynamic **C**lipping, which relaxes the clipping bound to encourage exploration range. Extensive experiments show that these mechanisms jointly improve performance, enabling our method to surpass state-of-the-art baselines. Code is available anonymously at https://github.com/Can-Lin/GUI-AC.

Preprint.

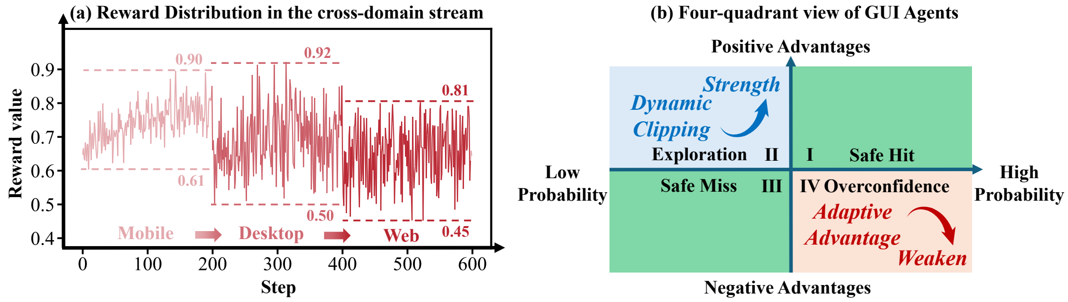

Figure 1: (a) The RFT-based method initially yields steady rewards. However, when transitioning to a new GUI domain, it shows sharp reward discontinuities and significant reward jumps at domain transitions. (b) Four-quadrant analysis of GUI-AC in the advantage-probability plane, where our method specifically targets the **Exploration** and **Overconfidence** regions.

1 Introduction

Graphical User Interfaces (GUIs) are the predominant means by which humans interact with digital devices [1]. However, unlike rigidly programmed automation, a powerful GUI agent must adapt to novel interface environments with human-like flexibility [2, 3, 4]. Moreover, GUI agents must continually operate on previously unseen interfaces in the wild, as the ever-evolving landscape of applications and design patterns introduces persistent distributional shifts over time, such as application updates, cross-platform migrations and changes in device resolution [5, 6, 7]. These continual distribution shifts pose significant challenges to the continual learning of GUI agents, which is formalized as Continual GUI Agents task [7].

GUI grounding is the fundamental capability of GUI agents to accurately map natural language instructions to pixel coordinates on interface elements [8, 9, 10]. In continual learning scenarios, reinforcement fine-tuning (RFT) exhibits natural advantages [7, 11]. GUI methods based on RFT leverage reward signals generated from online interactions, perform on-policy updates for self-correction, and thus achieve better grounding capability [12, 13]. However, standard RFT implicitly assumes a relatively stable interface distribution. When a GUI agent encounters unseen interface distributions after crossing task boundaries during continual learning, its grounding capability degrades significantly [7], manifesting as severe training instability [14, 15, 16, 17]. These issues are sharply magnified in continual learning for GUI agents, where distribution shifts occur frequently. As shown in Figure 1 (a), the reward signal becomes highly unstable after each domain transition, indicating a fundamental vulnerability of standard RFT under continual distribution shifts.

To dissect the root causes of grounding capability degradation in the Continual GUI Agents setting, we conduct a fine-grained analysis of policy dynamics. As shown in Figure 1(b), we map the agent's states onto the four quadrants defined by advantage and probability. A complete four-quadrant analysis is deferred to Section 2.2. The crux of the problem lies in two pivotal regions. Overconfidence corresponds to outdated high-confidence actions tied to the previous interface. These overconfident actionsare no longer aligned with the new GUI geometry, their rewards and gradients become highly inconsistent across rollouts. When the distribution changes, these predictions substantially amplify training noise and undermine localization stability. Exploration represents rare but correct exploratory actions under a novel interface. It constitutes the most critical source of effective gradients for adapting to the new distribution. However, the increase in its probability is obstructed by the fixed clipping bound. Under distribution shift, the impeded entropy increase in Exploration induce asymmetric dynamics, directly leading to rapid policy entropy collapse.

To address these challenges, we propose GUI-AC, a method that enhances the continual learning capability of GUI agents. We introduce grounding certainty as an observable proxy for grounding capability within each sample group. Guided by the grounding certainty, GUI-AC jointly calibrates two complementary mechanisms: (i) Adaptive **A**dvantage, which scales the normalized group advantage and down-weights noisy instances to prevent policy overconfidence; and (ii) Dynamic **C**lipping, which relaxes the clipping bound to selectively encourage the exploration range. These two mechanisms enable the policy to remain stable on previously seen tasks while actively exploring unseen interfaces. In summary, the main contributions of our paper are:

- We revisit grounding instability in continual GUI agents and for the first time employ a four-quadrant analysis to pinpoint the root causes of RFT failure.
- We propose GUI-AC, a novel method that enhances the continual learning capability of GUI agents through two core mechanisms: Adaptive Advantage and Dynamic Clipping.
- GUI-AC achieves state-of-the-art performance on the ScreenSpot-V1, V2, and Pro benchmarks, delivering robust improvements in training stability and continual generalization.

2 Method

2.1 Problem Formulation

The task of GUI grounding requires an agent to map a natural-language instruction and an interface to the pixel coordinates of the corresponding interactive element. Given a interface and an instruction, the model predicts a bounding box $\mathbf{b}^p = [x_1^p, y_1^p, x_2^p, y_2^p]$, with (x_1^p, y_1^p) and (x_2^p, y_2^p) denoting the top-left and bottom-right corners. The ground-truth box is $\mathbf{b}^{gt} = [x_1^{gt}, y_1^{gt}, x_2^{gt}, y_2^{gt}]$. We treat a prediction as correct if its center point $\mathbf{c}_p = \left(\frac{x_1^p + x_2^p}{2}, \frac{y_1^p + y_2^p}{2}\right)$ lies within $\mathbf{b}^{gt}$. We focus on the Continual GUI Agents setting [7], which is modeled as tasks that arrive sequentially, including a cross-domain sequence $\mathcal{T}_D = \{t_{d1}, t_{d2}, ..., t_{dn}\}$ and a cross-resolution sequence $\mathcal{T}_R = \{t_{r1}, t_{r2}, ..., t_{rn}\}$. The objective is to learn an policy that continually adapts to newly arriving tasks while preserving grounding performance on previously encountered tasks. Our method is built upon standard GRPO and the design of our reward function is presented in Appendix A.1.

2.2 Rethinking Continual GUI Agents in RFT

RFT-based methods are appealing for continual GUI agents due to online interaction and on-policy self-correction. However, under domain or resolution shifts, the same policy update can have qualitatively different effects on different sampled actions. To better understand this discrepancy, we rethink continual GUI agents through the joint lens of advantage and policy probability.

Four-Quadrant View of Continual GUI Agents. For a sampled action, its optimization effect is jointly determined by two factors: whether the action receives a positive or negative advantage, and whether the current policy assigns it a high or low probability. This yields four representative regions in the advantage-probability plane.

- **I: Safe Hit**. Actions with high probability and positive advantage. These actions correspond to interaction patterns that the policy has already mastered and are often inherited from seen interfaces.
- **II: Exploration**. Actions with low probability and positive advantage. These actions are rare but valuable. Under a shifted GUI distribution, only a few sampled actions accidentally hit the correct element. They provide the most important corrective signal for adapting to the new interface.
- **III: Safe Miss**. Actions with low probability and negative advantage. These actions are incorrect but usually less harmful to continual adaptation, since the policy already assigns them low probability.
- **IV: Overconfidence**. Actions with high probability and negative advantage. These actions are the most problematic under distribution shift. The policy confidently predicts coordinates based on an outdated interface layout, but these predictions no longer match the current GUI geometry.

This quadrant view reveals two coupled challenges. (i) Which advantages should be trusted when high-probability actions become overconfident? (ii) How can rare low-probability correct actions be explored before they disappear from future rollouts? These two challenges correspond to two failure modes of standard RFT in continual GUI grounding.

Unreliable Advantage Estimation. When the distribution shifts, the rollout outcomes within the same group often become highly imbalanced. Group-based advantage estimation is easily corrupted by noisy high-confidence failures, amplifying optimization variance. This effect is especially severe for Overconfidence. Common RFT control techniques cannot effectively address this issue in continual GUI agent scenarios. This limitation arises because these alternatives are essentially global stabilization methods: indiscriminate mechanisms applied at the parameter update level. We provide additional experiments and analysis in the Appendix A.2.

Fixed Clipping Causes Entropy Collapse. When the distribution shifts, only a very small number of samples within a rollout group typically hit the correct element by chance and receive high rewards. Specifically, the most valuable signals come from Exploration. However, the fixed clipping mechanism commonly used in standard RFT constrains the magnitude of probability-ratio changes between the new and old policies. For correct actions with initially low probability, their probability increase is tightly restricted, making it difficult for them to be sufficiently amplified within one or only a few updates. Over time, the policy becomes increasingly concentrated on a small set of already high-probability behaviors. This leads to a rapid decline in policy entropy during training and causes the policy to become prematurely deterministic.

Measuring Grounding Capability via Certainty. We introduce Grounding Certainty as an observable proxy for grounding capability, computed from rollout outcomes within each group. For each training instance i, we sample N rollouts from π_θ, obtaining rewards $\{r_{i,1}, \ldots, r_{i,N}\}$. We threshold the task feedback with a predefined threshold τ and define the grounding certainty as:

$$\hat{p}_i \;=\; \frac{1}{N}\sum_{g=1}^{N} \mathbb{I}[r_{i,g} \geq \tau]\,. \tag{1}$$

A high $\hat{p}_i$ indicates that the policy is taking safe hit actions. A low $\hat{p}_i$ indicates an uncertain or mismatched grounding regime. Such a group usually contains a mixture of exploratory and overconfident actions. Therefore, low $\hat{p}_i$ calls for two different treatments at the same time.

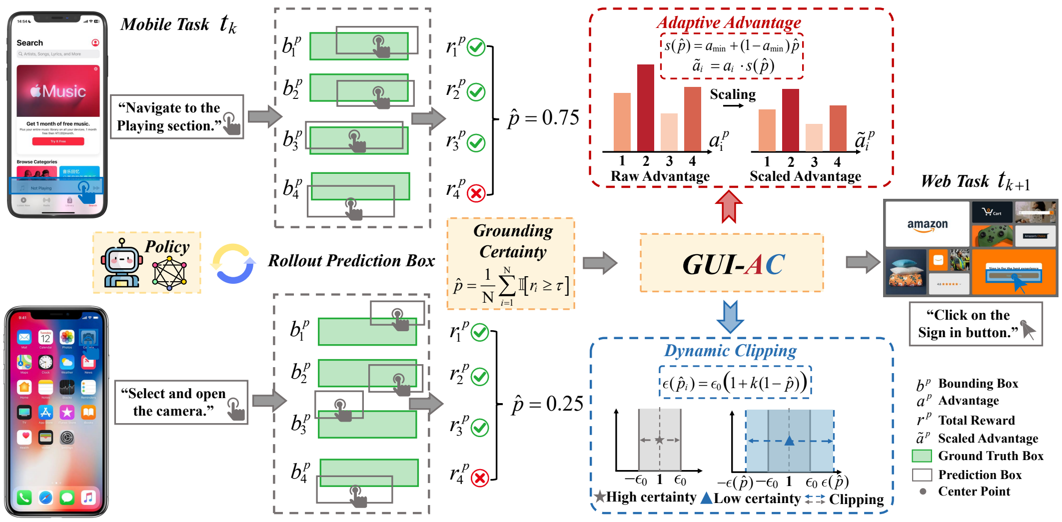

Figure 2: Illustration of GUI-AC. *Grounding Certainty* varies across GUI instances, motivating two complementary mechanisms: *Adaptive **A**dvantage*, which rescales advantages to down-weight overconfidenct rollouts, and *Dynamic **C**lipping*, which adjusts clipping bounds to balance exploration.

2.3 GUI-AC Framework

As illustrated in Figure 2, GUI-AC jointly calibrates two complementary mechanisms: (i) *Adaptive Advantage*, which scales the advantage to prevent policy overconfidence; (ii) *Dynamic Clipping*, which relaxes the clipping bound to encourage policy exploration.

Adaptive Advantage. In standard group-based normalization, all samples in the same rollout group contribute according to their normalized advantages. However, under distribution shift, low $\hat{p}_i$ groups often contain overconfident actions. These actions are no longer aligned with the new GUI interfaces.

We first compute the group-wise mean and standard deviation:

$$\mu_i = \frac{1}{N}\sum_{g=1}^{N} r_{i,g}, \qquad \sigma_i = \sqrt{\frac{1}{N}\sum_{g=1}^{N}(r_{i,g} - \mu_i)^2}. \tag{2}$$

To avoid division by zero when the rewards within a group are all equal, we define the normalized advantage with a small numerical stabilizer $\varepsilon > 0$:

$$A_{i,g} = \frac{r_{i,g} - \mu_i}{\sigma_i + \varepsilon}. \tag{3}$$

We define a certainty-conditioned scaling factor:

$$s(\hat{p}_i) = a_{\min} + (1 - a_{\min})\hat{p}_i, \tag{4}$$

where $a_{\min} \in (0, 1)$ lower-bounds the influence of low-certainty groups. The scaled advantage is:

$$\widetilde{A}_{i,g} = A_{i,g} \cdot s(\hat{p}_i). \tag{5}$$

When $\hat{p}_i$ is high, the rollout group is dominated by reliable Safe Hit actions, so $s(\hat{p}_i)$ approaches 1 and the original advantage signal is largely preserved. When $\hat{p}_i$ is low, the group is more likely to contain noisy overconfident failures, so the advantage magnitude is reduced.

Dynamic Clipping. Policy improvement relies on the repeated resampling of actions that receive positive feedback in the current batch, such that their probabilities can be progressively amplified. In low-certainty groups, the most valuable corrective actions often come from exploration. With a fixed clipping boundary, the probability increase of such actions is tightly capped.

Let $u_i = (s_i, x_i)$ be the input context and $c_{i,g,t} = (u_i, y_{i,g,<t})$ be the decoding context for the t-th token. The token-level probability ratio is defined as:

$$\rho_{i,g,t} = \frac{\pi_\theta(y_{i,g,t} \mid c_{i,g,t})}{\pi_{\text{ref}}(y_{i,g,t} \mid c_{i,g,t})}. \tag{6}$$

We define an uncertainty-conditioned clipping margin:

$$\epsilon(\hat{p}_i) = \epsilon_0\big(1 + k(1 - \hat{p}_i)\big), \tag{7}$$

where $k \geq 0$ is the trust-region expansion factor. The clipping bounds are:

$$U_i = 1 + \epsilon(\hat{p}_i), \qquad L_i = 1 - \epsilon(\hat{p}_i), \tag{8}$$

and the clipped ratio is:

$$\text{clip}_{\hat{p}_i}(\rho_{i,g,t}) = \min\big\{\max(\rho_{i,g,t},\, L_i),\, U_i\big\}. \tag{9}$$

When $\hat{p}_i$ is high, the clipping bound reduces to the standard conservative range, preventing unnecessary perturbation of already reliable Safe Hit actions. When $\hat{p}_i$ is low, the clipping bound becomes wider, allowing exploration actions to receive larger updates of the probability ratio.

Reinforcement Fine-tuning with GUI-AC. Following GRPO, we maximize a clipped surrogate objective and regularize policy updates via a token-level KL penalty. The clipped surrogate term is:

$$\ell_{i,g,t}(\theta) = \min\Big(\rho_{i,g,t}\, \widetilde{A}_{i,g},\ \text{clip}_{\hat{p}_i}(\rho_{i,g,t})\, \widetilde{A}_{i,g}\Big). \tag{10}$$

We compute the KL penalty under the same decoding context:

$$\text{KL}_{i,g,t} = D_{KL}(\pi_\theta(\cdot \mid c_{i,g,t}) \parallel \pi_{\text{ref}}(\cdot \mid c_{i,g,t}))\,. \tag{11}$$

The final objective is:

$$\mathcal{J}(\theta) = \mathbb{E}_{i,g,t}[\ell_{i,g,t}(\theta)] - \beta\, \text{KL}_{i,g,t}, \tag{12}$$

where $\beta \geq 0$ controls the strength of KL regularization.

3 Experiments

3.1 Experimental Setup

Dataset and Evaluation Benchmarks. We follow the continual learning evaluation protocol of Continual GUI Agents and evaluate under two sequential training regimes: (i) Cross-Domain transfer (Mobile $\rightarrow$ Desktop $\rightarrow$ Web) and (ii) Cross-Resolution transfer (Normal $\rightarrow$ High). We use Widget Captioning [5] for mobile applications, ShowUI-web [6] for standard-resolution desktop/web applications (mostly 1080p), and OmniACT [18] for high-resolution desktop/web applications with resolutions ranging from 1080p to 4K. For cross-domain continual learning, we train sequentially on Widget Captioning (Mobile) and then on OmniACT (Desktop and Web). For cross-resolution continual learning, we train from ShowUI-web (Normal) to OmniACT (High).

We evaluate on ScreenSpot-V1 (SSv1) [5], ScreenSpot-V2 (SSv2) [19], and ScreenSpot-Pro (SSPro) [20]. SSv1 and SSv2 span Mobile/Desktop/Web tasks and thus quantify cross-domain continual learning performance. SSPro is tailored to high-resolution software scenarios and comprises six interface categories: CAD, Development Programming (Dev), Creative Software (Creative), Scientific and Analytic (Scientific), Office Software (Office), and Operating System Commons (OS), which we use to assess cross-resolution transfer.

Baseline Methods and Implementation Details. We compare against representative RFT baselines: InfiGUI-R1 [21], SE-GUI [22], GUI-G^2 [23], and GUI-AiF [7]. InfiGUI-R1 uses the IoU between predicted and ground-truth bounding boxes as the primary feedback signal. SE-GUI uses the distance between box centers as the reward. GUI-G^2 models bounding boxes with a Gaussian distribution to construct a dense reward that jointly accounts for center deviation and region coverage. GUI-AiF improves robustness to domain and resolution shifts by incorporating point-level and region-level rewards. SE-GUI† denotes a hybrid variant that combines the IoU reward from InfiGUI-R1 with the distance reward from SE-GUI to improve training stability.

We adopt Qwen2.5VL-3B [24] as the vision–language backbone and use the same task format as GUI-AiF [7] for a controlled comparison. All models are trained on 4 $\times$ NVIDIA A100-80G GPUs with bfloat16; FlashAttention-2 and gradient checkpointing are enabled for efficiency. Unless otherwise stated, we use a learning rate of 1×10^{-6}, a global batch size of 8, and sample 4 candidate predictions per instruction for optimization. We set the KL regularization coefficient to $\beta = 0.04$. And we introduce two additional hyperparameters: a lower bound on advantage scaling $a_{\min} = 0.2$, and a trust-region expansion factor $k = 3.0$.

Table 1: Continual domain performance (%) of the proposed GUI-AC method on the ScreenSpot-V1 and ScreenSpot-V2 benchmark. The M. D. and W. denote the Mobile, Desktop and Web domain tasks, respectively. The upper bound corresponds to the state-of-the-art performance achieved by RFT-based methods for GUI agents and is provided as a reference.

Method		**SSv1 Accuracy (%)**							**SSv2 Accuracy (%)**						
		Mobile		Desktop		Web		**Avg.**	Mobile		Desktop		Web		**Avg.**
		Text	Icon	Text	Icon	Text	Icon		Text	Icon	Text	Icon	Text	Icon	
Proprietary Model															
GPT-4o		-	-	-	-	-	-	18.8	-	-	-	-	-	-	20.1
Open-source Model															
Qwen2.5VL-3B [24]		90.2	72.9	78.4	57.1	80.9	64.1	72.1	88.5	74.4	78.9	58.6	76.9	64.5	73.6
RFT-based GUI Model															
SE-GUI† [22]	Upper Bound	-	-	-	-	-	-	88.2	-	-	-	-	-	-	90.3
	M.	91.5	75.9	72.9	59.3	73.9	55.1	71.4	90.4	81.0	84.1	60.1	71.8	64.5	75.3
	M.→D.	91.8	76.6	78.1	65.0	76.3	58.3	74.4	92.2	81.9	87.6	67.4	73.7	63.5	77.7
	M.→D.→W.	92.0	78.2	83.1	65.7	77.4	60.2	76.1	93.2	82.5	90.2	69.3	75.1	63.1	78.9
GUI-G^2 [23]	Upper Bound	96.7	90.8	95.9	88.6	90.9	86.9	92.0	-	-	-	-	-	-	93.3
	M.	91.9	77.2	78.8	61.4	72.6	51.2	72.2	92.5	81.5	85.2	66.4	73.5	64.8	77.3
	M.→D.	93.8	76.4	88.2	61.4	75.2	53.9	74.8	94.2	78.7	87.2	66.4	77.8	65.7	78.3
	M.→D.→W.	95.2	76.4	92.8	63.6	79.6	54.9	77.1	94.8	79.1	89.3	68.6	81.2	68.6	80.3
GUI-AiF [7]	M.	92.9	78.7	81.9	68.6	81.6	65.1	78.1	93.3	82.1	88.5	73.6	80.3	68.0	80.9
	M.→D.	94.1	77.3	93.2	69.3	81.9	66.2	80.3	95.9	79.6	89.7	77.1	81.2	68.3	81.9
	M.→D.→W.	96.1	76.9	**95.7**	68.3	84.8	68.2	81.7	96.6	**83.6**	90.2	**77.4**	82.9	70.5	83.5
GUI-AC (Ours)	M.	94.1	81.2	86.6	70.0	82.2	70.0	80.7	95.1	82.1	89.2	75.0	82.5	68.5	82.1
	M.→D.	95.2	81.2	92.2	70.7	87.8	69.0	82.7	96.5	82.5	92.3	77.1	83.7	68.5	83.4
	M.→D.→W.	**96.5**	**82.4**	94.3	**71.4**	**88.7**	**70.9**	**84.0**	**97.6**	82.5	**95.4**	77.1	**86.3**	**71.4**	**85.1**

3.2 Main Results

Continual GUI Domain. We first evaluate the continual-domain performance of GUI-AC. Table 1 shows that GUI-AC delivers consistent improvements under continual domain shifts. In line with prior observations in continual learning of GUI agents, proprietary general-purpose models exhibit low grounding accuracy. Open-source VLM backbones provide a stronger starting point, but still leave substantial headroom. Among adaptation strategies, GUI-AC achieves the highest accuracy, with stable gains for both text-based and icon-based interactions across both SSv1 and SSv2. Importantly, performance accumulates over time: as training proceeds along the cross-domain stream, average accuracy increases steadily, suggesting that learning on later domains does not substantially erode earlier grounding skills and instead yields net improvements across domains.

Table 2: Continual resolution performance (%) of the proposed GUI-AC method on the ScreenSpot-Pro benchmark. The N. and H. denote the Normal and High resolution tasks, respectively. The upper bound is corresponds to the state-of-the-art performance achieved by RFT-based methods for GUI agents and is provided as a reference.

Method		**SSPro Accuracy (%)**												**Avg.**
		CAD		Dev		Creative		Scientific		Office		OS		
		Text	Icon	Text	Icon	Text	Icon	Text	Icon	Text	Icon	Text	Icon	
Proprietary Model														
GPT-4o		2.0	0.0	1.3	0.0	1.0	0.0	2.1	0.0	1.1	0.0	0.0	0.0	0.8
Claude Computer Use		14.5	3.7	22.0	3.9	25.9	3.4	33.9	15.8	30.1	16.3	11.0	4.5	17.1
Open-source Model														
Qwen2.5VL-3B [24]		9.1	7.3	22.1	1.4	26.8	2.1	38.2	7.3	33.9	15.1	10.3	1.1	16.1
RFT-based GUI Methods														
	Upper Bound	55.8	12.5	68.8	17.2	57.1	15.4	77.1	24.5	74.0	32.7	57.9	21.3	47.5
GUI-G^2 [23]	N.	24.3	1.7	17.5	2.1	20.2	6.3	22.2	12.8	36.2	11.3	17.7	4.5	14.7
	N.→H.	24.5	6.3	24.4	4.1	23.7	**5.6**	27.3	12.6	35.1	9.4	21.2	5.6	16.7
GUI-AiF [7]	N.	27.4	4.7	24.7	1.4	17.7	8.2	29.2	14.6	35.3	20.8	17.8	10.1	17.7
	N.→H.	20.3	**15.6**	29.2	2.1	23.2	3.5	33.3	14.7	38.4	**18.2**	22.4	6.8	19.0
GUI-AC (Ours)	N.	25.4	3.1	29.2	2.1	32.3	2.1	36.8	12.6	49.2	13.2	22.4	4.5	19.4
	N.→H.	**27.9**	7.3	**39.6**	**4.1**	**36.4**	**5.6**	**41.0**	**15.5**	**49.7**	17.0	**26.2**	**7.9**	**23.1**

Continual GUI Resolution. We then evaluate the continual-resolution performance of GUI-AC. Table 2 further indicates that GUI-AC is robust to continual resolution shifts and exhibits clear forward transfer. Since the Gaussian modeling-based RFT provides better performance, we compare against GUI-G^2 and GUI-AiF as representative RFT baselines throughout this evaluations. Overall, GUI-AC performs best under continual resolution shifts. After continual training on high-resolution data, accuracy improves across all categories, with the largest gains on text targets. Across methods, icon grounding remains more difficult than text grounding, likely because small icons provide limited visual evidence in dense professional interfaces. Nevertheless, GUI-AC consistently improves icon performance as well, demonstrating stronger robustness under continual resolution variation.

Table 3: Ablation study of GUI-AC on ScreenSpot-V1 and ScreenSpot-V2 benchmark. The M. D. and W. denote the Mobile, Desktop and Web domain tasks, respectively.

Method		**SSv1 Accuracy (%)**						**Avg.**	**SSv2 Accuracy (%)**						**Avg.**
		Mobile		Desktop		Web			Mobile		Desktop		Web		
		Text	Icon	Text	Icon	Text	Icon		Text	Icon	Text	Icon	Text	Icon	
GUI-AC	M.	94.1	81.2	86.6	70.0	82.2	70.0	80.7	95.1	82.1	89.2	75.0	82.5	68.5	82.1
	M.→D.	95.2	81.2	92.2	70.7	87.8	69.0	82.7	96.5	82.5	92.3	77.1	83.7	68.5	83.4
	M.→D.→W.	**96.5**	**82.4**	**94.3**	**71.4**	**88.7**	**70.9**	**84.0**	**97.6**	**82.5**	**95.4**	**77.1**	**86.3**	**71.4**	**85.1**
GUI-A	M.	93.8	81.7	86.1	64.3	83.3	68.2	79.6	94.8	83.9	87.1	68.6	84.1	70.9	81.6
	M.→D.	94.1	80.6	90.2	68.6	84.2	70.0	81.2	95.9	81.7	91.8	73.6	81.2	68.6	82.1
	M.→D.→W.	95.2	78.6	90.7	68.6	84.9	70.7	81.5	97.2	80.1	91.2	75.0	81.2	70.1	82.5
GUI-C	M.	94.1	80.6	88.2	63.6	82.2	68.6	79.6	95.5	82.1	88.4	70.0	83.8	70.1	81.7
	M.→D.	95.6	81.2	91.2	66.4	84.8	69.0	81.4	96.2	81.5	92.3	73.6	81.2	68.3	82.2
	M.→D.→W.	96.3	81.6	92.3	68.6	86.2	69.0	82.3	97.2	81.0	91.8	73.6	83.7	68.5	82.6

3.3 Ablation Study

Certainty-Calibrated Components. We conduct an ablation study of GUI-AC. As shown in Table 3. We isolate the two certainty-calibrated components by activating either Adaptive Advantage (GUI-A) or Dynamic Clipping (GUI-C) alone. Overall, the full GUI-AC yields the highest average accuracy, while removing either component consistently degrades performance, indicating that the two mechanisms provide complementary benefits. Moreover, GUI-C achieves a higher final average accuracy than GUI-A on both SSV1 and SSV2, implying that Dynamic Clipping is the primary driver of the improvement. These results suggest that fixed clipping imposes a stronger constraint on exploration, and that alleviating this constraint is crucial for performance.

Hyperparameter Sensitivity. We further analyze the sensitivity of GUI-AC to two key hyperparameters: $a_{\min}$ and k. Figure 3 summarizes the final average accuracy across different $(a_{\min}, k)$ configurations. Notably, the majority of parameter configurations still yield better performance than

GUI-AIF. Among the values tested, the default choice $(0.2, 3.0)$ performs best on all benchmarks. A larger $a_{\min}$ attenuates the down-weighting of uncertain samples and an excessively small $a_{\min}$ makes policy updates overly conservative. The expansion factor k exhibits a similar trade-off. If k is too small, the trust region is not sufficiently enlarged under uncertainty and if k is too large, the resulting permissive exploration can exacerbate noisy credit assignment. Taken together, GUI-AC remains stable under moderate hyperparameter variations but benefits from a balanced choice of $(a_{\min}, k)$.

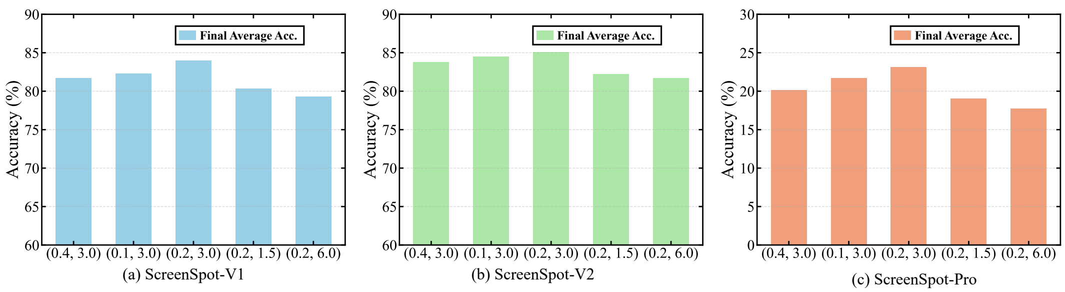

Figure 3: Hyperparameter sensitivity analysis for $a_{\min}$ and k. Performance peaks at $(a_{\min} = 0.2,\ k = 3.0)$ on ScreenSpot-V1, ScreenSpot-V2 and ScreenSpot-Pro benchmarks.

Table 4: Grounding certainty sensitivity study to the threshold on ScreenSpot-V1 and ScreenSpot-V2 benchmark. The M. D. and W. denote the Mobile, Desktop and Web domain tasks, respectively.

Method		**SSv1 Accuracy (%)**							**SSv2 Accuracy (%)**						
		Mobile		Desktop		Web		**Avg.**	Mobile		Desktop		Web		**Avg.**
		Text	Icon	Text	Icon	Text	Icon		Text	Icon	Text	Icon	Text	Icon	
$\tau = 0.6$	M.	94.1	81.2	86.6	70.0	82.2	70.0	80.7	95.1	82.1	89.2	75.0	82.5	68.5	82.1
	M.→D.	95.2	81.2	92.2	70.7	87.8	69.0	82.7	96.5	82.5	92.3	77.1	83.7	68.5	83.4
	M.→D.→W.	**96.5**	**82.4**	**94.3**	**71.4**	**88.7**	**70.9**	**84.0**	**97.6**	**82.5**	**95.4**	**77.1**	**86.3**	**71.4**	**85.1**
$\tau = 0.4$	M.	94.0	81.0	86.1	69.3	81.8	69.4	80.3	94.9	81.8	88.8	74.3	82.0	68.0	81.6
	M.→D.	95.0	80.9	91.6	70.0	87.0	68.2	82.1	96.2	82.1	91.8	76.4	83.0	68.0	82.9
	M.→D.→W.	96.2	81.9	93.8	70.8	88.0	70.2	83.5	97.3	82.1	94.8	76.6	85.4	70.8	84.5
$\tau = 0.8$	M.	93.6	80.4	85.3	68.7	80.9	68.5	79.6	94.4	81.3	87.9	73.5	81.1	67.0	80.9
	M.→D.	94.7	80.2	90.9	69.2	85.9	67.4	81.4	95.8	81.6	90.8	75.6	82.1	67.2	82.2
	M.→D.→W.	95.8	81.0	92.9	70.0	86.7	69.1	82.6	96.8	81.8	93.6	76.0	84.6	69.8	83.8

Grounding Certainty Sensitivity. We further evaluate the sensitivity to the threshold τ by testing additional values (0.4 and 0.8) alongside the default 0.6. The results, as shown in Table 4, both 0.4 and 0.8 underperform compared to 0.6, indicating that a moderate threshold achieves a better balance between filtering noisy signals and preserving useful training samples.

3.4 Discussion

Grounding Certainty Distribution Analysis. Figure 4 characterizes how grounding certainty evolves under cross-domain continual learning by comparing GUI-AiF and GUI-AC. Because we sample $n = 4$ candidates per instruction, the certainty score is quantized to $\{0, 0.25, 0.5, 0.75, 1\}$. In the Mobile stage, the distribution places most of its mass in high-certainty bins, suggesting stable grounding when the domain is unchanged. As the interface shifts from Mobile to Desktop and further to Web, the distribution progressively rebalances toward lower-certainty bins, indicating growing uncertainty induced by domain drift. This effect is substantially stronger for GUI-AiF. In contrast, GUI-AC exhibits a gentler redistribution and retains more high-certainty mass, implying improved robustness of grounding under continual domain shifts.

Reward Distribution Analysis. Figure 5 presents violin plots of stage-wise reward distributions to assess optimization stability under continual GUI shifts. Each violin summarizes the distribution of per-rollout rewards within a stage. We observe consistent patterns in both the cross-domain and cross-resolution evaluations. Under GUI-AiF, rewards become increasingly dispersed and develop a pronounced lower tail after each transfer, suggesting deteriorating stability. By contrast, GUI-AC yields markedly more stable distributions across the stream: it suppresses tail growth, limits

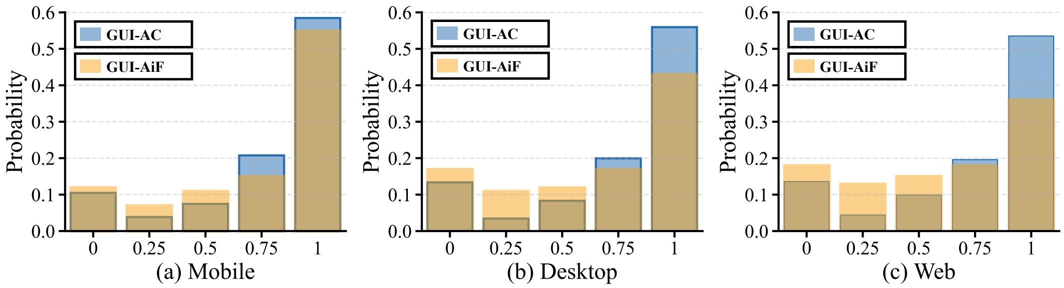

Figure 4: Histograms of grounding certainty under $n = 4$ sampled candidates per instruction.

distributional widening after shifts, and preserves the bulk of the reward mass. Overall, these results support that GUI-AC enhances training stability for continual learning of GUI agents.

Visualization Comparison. Figure 6 compares interaction-region heatmaps in terms of spatial alignment and concentration. Figure 6a displays the Web interface with the instruction "View more details about the sword and shield item". The open-source baseline shows weak, poorly aligned activation, where the peak often shifts to nearby background regions (Fig. 6b). GUI-AiF amplifies target-related activation, yet retains noticeable spillover to adjacent candidates (Fig. 6c). By contrast, GUI-AC produces a sharply localized peak on the correct item with minimal surrounding mass (Fig. 6d), indicating more accurate and robust GUI grounding.

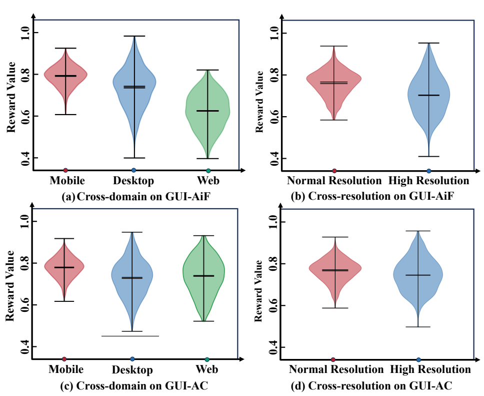

Figure 5: Reward distribution under continual learning of GUI agents.(a)/(c) under cross-domain. (b)/(d) under cross-resolution.

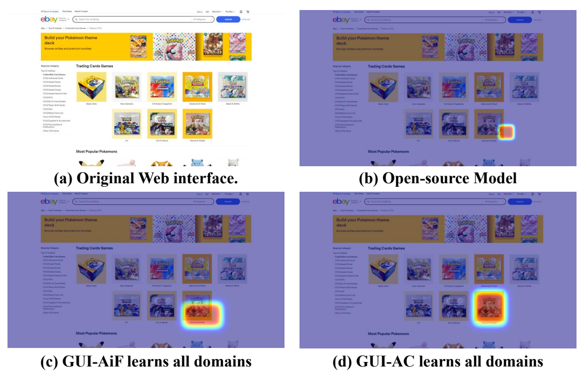

Figure 6: Visualizations of interaction region heatmaps on web platforms. Instruction used: *"View more details about the sword and shield item."*

4 Related Works

4.1 GUI Agents

GUI agents are designed to understand natural language instructions and perform corresponding low-level interactions (e.g., click, type, swipe) to accomplish user tasks on digital screens [2, 3, 5, 25, 26, 27, 28, 29, 30]. Existing methods for GUI agents and grounding can be broadly categorized into two paradigms. (1) Expert design-driven workflow paradigm. This line of work typically uses proprietary multimodal models as high-level planners, and combines them with manually engineered planners [31, 32] or grounders [6, 25, 33, 19] to form a controllable pipeline. Grounding is commonly achieved either by leveraging structural signals [34, 35] or by using vision-based tooling [36, 37, 38]. Workflow approaches are easy to deploy and interpret, yet often rely on external modules and hand-crafted rules, which can reduce robustness under domain shift and UI drift [39]. (2) Data-driven training paradigm. Another line of work improves GUI perception and reasoning by scaling task-specific data or architectures [3, 40, 39]. Although scaling data can strengthen performance, these methods may still face generalization challenges when encountering entirely new interface styles [41].

4.2 SFT vs. RFT for Continual Post-training

To adapt a base model to specific grounding tasks, two mainstream fine-tuning approaches are used: Supervised Fine-Tuning (SFT) and Reinforcement Fine-Tuning (RFT). SFT is prone to overfitting the current label distribution and can suffer from forgetting under distribution shifts [42, 43, 44]. RFT, particularly with on-policy updates, is better suited for continual improvement but faces optimization challenges when the interface evolves [12, 13, 45]. Recent work has introduced verifiable rewards into GUI tasks, such as the R1 paradigm [46] adapted for GUI interaction [41, 47, 21, 48]. However, many designs offer sparse or point-level supervision, providing limited geometric guidance. While methods such as GUI-G^2 [23] improve optimization efficiency in relatively stable settings, mechanisms that explicitly promote forward transfer across a stream of changing GUIs remain underexplored. GUI-AiF [7] explicitly addresses continual adaptation via flux-oriented reward shaping, yet it still fails to effectively address the noisy reward and entropy collapse issues.

5 Conclusion

In this paper, we further investigated the continual learning of GUI Agents. We reveal the limitations of unreliable advantage estimation and fixed clipping under non-stationary GUI data. Then, we proposed GUI-AC, which leverages adaptive advantage and dynamic clipping to enhance continual learning performance of GUI agents. Extensive experiments demonstrate that GUI-AC consistently adapts to varying GUI domain and resolutions, outperforming baselines in dynamic GUI environments.

References

[1] Shuai Wang, Weiwen Liu, Jingxuan Chen, Yuqi Zhou, Weinan Gan, Xingshan Zeng, Yuhan Che, Shuai Yu, Xinlong Hao, Kun Shao, et al. GUI Agents with Foundation Models: A Comprehensive Survey. *arXiv preprint arXiv:2411.04890*, 2024.

[2] Fei Tang, Haolei Xu, Hang Zhang, Siqi Chen, Xingyu Wu, Yongliang Shen, Wenqi Zhang, Guiyang Hou, Zeqi Tan, Yuchen Yan, et al. A Survey on (M)LLM-Based GUI Agents. *arXiv preprint arXiv:2504.13865*, 2025.

[3] Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. CogAgent: A Visual Language Model for GUI Agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14281–14290, 2024.

[4] Ziyi Guan, Jason Chun Lok Li, Zhijian Hou, Pingping Zhang, Donglai Xu, Yuzhi Zhao, Mengyang Wu, Jinpeng Chen, Thanh-Toan Nguyen, Pengfei Xian, et al. KG-RAG: Enhancing GUI Agent Decision-Making via Knowledge Graph-Driven Retrieval-Augmented Generation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 5396–5405, 2025.

[5] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9313–9332, 2024.

[6] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. ShowUI: One Vision-Language-Action Model for GUI Visual Agent. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 19498–19508, 2025.

[7] Ziwei Liu, Borui Kang, Hangjie Yuan, Zixiang Zhao, Wei Li, Yifan Zhu, and Tao Feng. Continual GUI Agents. *arXiv*, 2026.

[8] Tao Feng, Mang Wang, and Hangjie Yuan. Overcoming Catastrophic Forgetting in Incremental Object Detection via Elastic Response Distillation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9427–9436, 2022.

[9] Yong Du, Yuchen Yan, Fei Tang, Zhengxi Lu, Chang Zong, Weiming Lu, Shengpei Jiang, and Yongliang Shen. Test-Time Reinforcement Learning for GUI Grounding via Region Consistency. *arXiv preprint arXiv:2508.05615*, 2025.

[10] Xianhang Ye, Yiqing Li, Wei Dai, Miancan Liu, Ziyuan Chen, Zhangye Han, Hongbo Min, Jinkui Ren, Xiantao Zhang, Wen Yang, et al. GUI-ARP: Enhancing Grounding with Adaptive Region Perception for GUI Agents. *arXiv preprint arXiv:2509.15532*, 2025.

[11] Shuquan Lian, Yuhang Wu, Jia Ma, Yifan Ding, Zihan Song, Bingqi Chen, Xiawu Zheng, and Hui Li. UI-AGILE: Advancing GUI Agents with Effective Reinforcement Learning and Precise Inference-Time Grounding. *arXiv preprint arXiv:2507.22025*, 2025.

[12] Ziyu Liu, Zeyi Sun, Yuhang Zang, Xiaoyi Dong, Yuhang Cao, Haodong Duan, Dahua Lin, and Jiaqi Wang. Visual-RFT: Visual Reinforcement Fine-Tuning. *arXiv preprint arXiv:2503.01785*, 2025.

[13] Zhihao Zhang, Qiaole Dong, Qi Zhang, Jun Zhao, Enyu Zhou, Zhiheng Xi, Senjie Jin, Xiaoran Fan, Yuhao Zhou, Mingqi Wu, et al. Why Reinforcement Fine-Tuning Enables MLLMs Preserve Prior Knowledge Better: A Data Perspective. *arXiv preprint arXiv:2506.23508*, 2025.

[14] Jinpeng Wang, Chao Li, Ting Ye, Mengyuan Zhang, Wei Liu, and Jian Luan. ICPO: Intrinsic Confidence-Driven Group Relative Preference Optimization for Efficient Reinforcement Learning. *arXiv preprint arXiv:2511.21005*, 2025.

[15] Shihui Yang, Chengfeng Dou, Peidong Guo, Kai Lu, Qiang Ju, Fei Deng, and Rihui Xin. DCPO: Dynamic Clipping Policy Optimization. *arXiv preprint arXiv:2509.02333*, 2025.

[16] Zhipeng Chen, Yingqian Min, Beichen Zhang, Jie Chen, Jinhao Jiang, Daixuan Cheng, Wayne Xin Zhao, Zheng Liu, Xu Miao, Yang Lu, et al. An Empirical Study on Eliciting and Improving R1-like Reasoning Models. *arXiv preprint arXiv:2503.04548*, 2025.

[17] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. *arXiv preprint arXiv:2503.14476*, 2025.

[18] Raghav Kapoor, Yash Parag Butala, Melisa Russak, Jing Yu Koh, Kiran Kamble, Waseem AlShikh, and Ruslan Salakhutdinov. OmniACT: A Dataset and Benchmark for Enabling Multimodal Generalist Autonomous Agents for Desktop and Web. In *European Conference on Computer Vision*, pages 161–178. Springer, 2024.

[19] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. OS-ATLAS: A Foundation Action Model for Generalist GUI Agents. *arXiv preprint arXiv:2410.23218*, 2024.

[20] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. ScreenSpot-Pro: GUI Grounding for Professional High-Resolution Computer Use. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 8778–8786, 2025.

[21] Yuhang Liu, Pengxiang Li, Congkai Xie, Xavier Hu, Xiaotian Han, Shengyu Zhang, Hongxia Yang, and Fei Wu. InfiGUI-R1: Advancing Multimodal GUI Agents from Reactive Actors to Deliberative Reasoners. *arXiv preprint arXiv:2504.14239*, 2025.

[22] Xinbin Yuan, Jian Zhang, Kaixin Li, Zhuoxuan Cai, Lujian Yao, Jie Chen, Enguang Wang, Qibin Hou, Jinwei Chen, Peng-Tao Jiang, et al. Enhancing Visual Grounding for GUI Agents via Self-Evolutionary Reinforcement Learning. *arXiv preprint arXiv:2505.12370*, 2025.

[23] Fei Tang, Zhangxuan Gu, Zhengxi Lu, Xuyang Liu, Shuheng Shen, Changhua Meng, Wen Wang, Wenqi Zhang, Yongliang Shen, Weiming Lu, et al. GUI-G^2: Gaussian Reward Modeling for GUI Grounding. *arXiv preprint arXiv:2507.15846*, 2025.

[24] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2.5-VL Technical Report. *arXiv preprint arXiv:2502.13923*, 2025.

[25] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the Digital World as Humans Do: Universal Visual Grounding for GUI Agents. *arXiv preprint arXiv:2410.05243*, 2024.

[26] Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, et al. Large Language Model-Brained GUI Agents: A Survey. *arXiv preprint arXiv:2411.18279*, 2024.

[27] Qiushi Sun, Kanzhi Cheng, Zichen Ding, Chuanyang Jin, Yian Wang, Fangzhi Xu, Zhenyu Wu, Chengyou Jia, Liheng Chen, Zhoumianze Liu, et al. OS-Genesis: Automating GUI Agent Trajectory Construction via Reverse Task Synthesis. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5555–5579, 2025.

[28] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. *Advances in Neural Information Processing Systems*, 36:38154–38180, 2023.

[29] Yuhao Yang, Yue Wang, Dongxu Li, Ziyang Luo, Bei Chen, Chao Huang, and Junnan Li. Aria-UI: Visual Grounding for GUI Instructions. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 22418–22433, 2025.

[30] Longxi Gao, Li Zhang, and Mengwei Xu. UIShift: Enhancing VLM-based GUI Agents through Self-supervised Reinforcement Learning. *arXiv preprint arXiv:2505.12493*, 2025.

[31] Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception. *arXiv preprint arXiv:2401.16158*, 2024.

[32] Chaoyun Zhang, Liqun Li, Shilin He, Xu Zhang, Bo Qiao, Si Qin, Minghua Ma, Yu Kang, Qingwei Lin, Saravan Rajmohan, et al. UFO: A UI-Focused Agent for Windows OS Interaction. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 597–622, 2025.

[33] Xiao Liu, Bo Qin, Dongzhu Liang, Guang Dong, Hanyu Lai, Hanchen Zhang, Hanlin Zhao, Iat Long Iong, Jiadai Sun, Jiaqi Wang, et al. AutoGLM: Autonomous Foundation Agents for GUIs. *arXiv preprint arXiv:2411.00820*, 2024.

[34] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Android in the Wild: A Large-Scale Dataset for Android Device Control. *Advances in Neural Information Processing Systems*, 36:59708–59728, 2023.

[35] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. AppAgent: Multimodal Agents as Smartphone Users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–20, 2025.

[36] Yuning Du, Chenxia Li, Ruoyu Guo, Xiaoting Yin, Weiwei Liu, Jun Zhou, Yifan Bai, Zilin Yu, Yehua Yang, Qingqing Dang, et al. PP-OCR: A Practical Ultra Lightweight OCR System. *arXiv preprint arXiv:2009.09941*, 2020.

[37] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment Anything. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4015–4026, 2023.

[38] Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. OmniParser for Pure Vision Based GUI Agent. *arXiv preprint arXiv:2408.00203*, 2024.

[39] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. *arXiv preprint arXiv:2501.12326*, 2025.

[40] Gilles Baechler, Srinivas Sunkara, Maria Wang, Fedir Zubach, Hassan Mansoor, Vincent Etter, Victor Cărbune, Jason Lin, Jindong Chen, and Abhanshu Sharma. ScreenAI: A Vision-Language Model for UI and Infographics Understanding. *arXiv preprint arXiv:2402.04615*, 2024.

[41] Run Luo, Lu Wang, Wanwei He, Longze Chen, Jiaming Li, and Xiaobo Xia. GUI-R1: A Generalist R1-Style Vision-Language Action Model For GUI Agents. *arXiv preprint arXiv:2504.10458*, 2025.

[42] Haiyang Guo, Fanhu Zeng, Fei Zhu, Jiayi Wang, Xukai Wang, Jingang Zhou, Hongbo Zhao, Wenzhuo Liu, Shijie Ma, Da-Han Wang, Xu-Yao Zhang, and Cheng-Lin Liu. Continual Learning for Generative AI: From LLMs to MLLMs and Beyond, 2025.

[43] Wenzhuo Liu, Fei Zhu, Haiyang Guo, Longhui Wei, and Cheng-Lin Liu. LLaVA-c: Continual Improved Visual Instruction Tuning. *arXiv preprint arXiv:2506.08666*, 2025.

[44] Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. RL's Razor: Why Online Reinforcement Learning Forgets Less. *arXiv preprint arXiv:2509.04259*, 2025.

[45] Hangzhan Jin, Sitao Luan, Sicheng Lyu, Guillaume Rabusseau, Reihaneh Rabbany, Doina Precup, and Mohammad Hamdaqa. RL Fine-Tuning Heals OOD Forgetting in SFT. *arXiv preprint arXiv:2509.12235*, 2025.

[46] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948*, 2025.

[47] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Guanjing Xiong, and Hongsheng Li. UI-R1: Enhancing Efficient Action Prediction of GUI Agents by Reinforcement Learning. *arXiv preprint arXiv:2503.21620*, 2025.

[48] Yuqi Zhou, Sunhao Dai, Shuai Wang, Kaiwen Zhou, Qinglin Jia, and Jun Xu. GUI-G1: Understanding R1-Zero-Like Training for Visual Grounding in GUI Agents. *arXiv preprint arXiv:2505.15810*, 2025.

A Appendix

A.1 Reward Function and Its Interaction with the Method.

In our implementation, GUI-AC uses the same task format as Continual GUI Agents [7]. This ensures a controlled comparison.

Given N predicted bounding boxes $\{b_1^p, b_2^p, \ldots, b_N^p\}$ for the same instruction, where $b_i^p = [x_{1,i}^p,\ y_{1,i}^p,\ x_{2,i}^p,\ y_{2,i}^p]$, we compute the corresponding center point as $c_i^p = \left(\frac{x_{1,i}^p + x_{2,i}^p}{2},\ \frac{y_{1,i}^p + y_{2,i}^p}{2}\right)$. Conceptually, each $\mathbf{c}_i^p$ serves as a candidate anchor point proposed by the agent given its understanding of the instruction. Let the centroid be $\bar{c}^p = \frac{1}{N}\sum_{j=1}^{N} c_j^p$. We define the point-diversity reward as the empirical spatial variance of the predicted centers:

$$R_p = \frac{1}{N}\sum_{i=1}^{N} \left\| c_i^p - \bar{c}^p \right\|^2 . \tag{13}$$

A larger R_p indicates that the predicted centers are more dispersed across the interface, instead of collapsing to a single location, thereby improving robustness to varying interaction positions under GUI flux.

Point diversity alone does not ensure robustness to scale variation, since GUI elements may change in size while remaining semantically aligned. We therefore additionally encourage diversity in the predicted extents by modeling each predicted region as a Gaussian $\mathcal{N}_i(\mu_i, \Sigma_i)$ and measuring separation via the Bhattacharyya distance:

$$D_B(\mathcal{N}_i, \mathcal{N}_j) = \frac{1}{8}(\mu_i - \mu_j)^T \Sigma_{\text{avg}}^{-1}(\mu_i - \mu_j) + \frac{1}{2}\ln\left(\frac{\det(\Sigma_{\text{avg}})}{\sqrt{\det(\Sigma_i)\det(\Sigma_j)}}\right), \tag{14}$$

where $\Sigma_{\text{avg}} = \frac{\Sigma_i + \Sigma_j}{2}$. We define the region-level reward as the average pairwise distance across the N predictions:

$$R_r = \frac{2}{N(N-1)}\sum_{i=1}^{N-1}\sum_{j=i+1}^{N} D_B(\mathcal{N}_i, \mathcal{N}_j). \tag{15}$$

Maximizing R_r encourages the agent to produce spatially separated regions, improving robustness to the diverse element scales encountered in fluxional GUIs.

Finally, we combine the two shaping signals as

$$R = \alpha R_p + \gamma R_r, \tag{16}$$

where α and γ control the relative strength of point-level exploration and region-level separation, respectively.

The two patterns discussed in our paper match the two geometric modes emphasized by this reward. In this sense, grounding certainty is not a detached heuristic. It serves as a proxy for whether the rollout group has formed a consistent geometric grounding mode under the same reward.

A.2 The Comparison with RL Baselines.

We have added the following controls on top of GUI-AiF: (i) larger rollout group size ($N = 8$), (ii) lower learning rate ($r = 5 \times 10^{-7}$), and (iii) stronger gradient clipping ($\epsilon = 0.4$). The results indicate that although these three control methods yield improvements in only certain performance metrics and fail to achieve comprehensive enhancement, none can match the performance of GUI-AC.

More importantly, we do not expect these controls to fully replace GUI-AC in the Continual GUI Agents setting. The problem we face is not merely one of optimization instability in the general sense. GUI interface elements possess inherent 2D spatial attributes, where the predicted bounding box determines both the interaction location and the target coverage. GUI grounding is inherently a continuous spatial problem. Under the continual learning setting, points and regions shift in domain or resolution, thereby repeatedly triggering instability during task switching. Overall, this problem exhibits both strong sample dependence and geometric characteristics. In other words, the true source

Table 5: Continual domain performance (%) of the RL baselines on the ScreenSpot-V1 and ScreenSpot-V2 benchmark. The M. D. and W. denote the Mobile, Desktop and Web domain tasks, respectively.

Method		**SSv1 Accuracy (%)**							**SSv2 Accuracy (%)**						
		Mobile		Desktop		Web		**Avg.**	Mobile		Desktop		Web		**Avg.**
		Text	Icon	Text	Icon	Text	Icon		Text	Icon	Text	Icon	Text	Icon	
$N = 8$	M.	94.5	79.5	88.1	65.0	81.6	63.6	78.7	95.2	80.1	88.1	71.7	82.1	65.5	80.5
	M.→D.	95.2	80.3	89.2	65.6	82.9	65.1	79.7	96.6	81.0	89.6	71.0	80.3	68.0	81.1
	M.→D.→W.	95.6	79.0	92.7	66.4	84.8	68.6	81.2	96.9	81.9	93.8	72.4	84.2	68.3	82.9
$r = 5 \times 10^{-7}$	M.	94.3	78.8	87.5	64.2	80.9	62.8	78.1	94.8	79.5	87.3	70.8	81.4	64.8	79.8
	M.→D.	94.8	79.5	88.6	64.8	82.1	64.2	79.0	96.2	80.3	88.9	70.2	79.5	67.3	80.4
	M.→D.→W.	95.2	78.2	91.9	65.6	84.0	67.5	80.4	96.5	81.0	92.9	71.4	83.4	67.5	82.1
$\epsilon = 0.4$	M.	94.0	78.0	86.8	63.5	80.0	61.8	77.4	94.3	78.8	86.4	70.0	80.5	63.8	79.0
	M.→D.	94.5	78.8	87.9	64.0	81.2	63.2	78.3	95.8	79.5	88.1	69.4	78.8	66.5	79.7
	M.→D.→W.	94.9	77.5	91.2	64.8	83.2	66.4	79.7	96.1	80.1	92.1	70.4	82.5	66.8	81.3

of instability does not lie in the average behavior of all samples, but in specific groups of samples that exhibit geometric inconsistency under distribution shift. Consequently, while global stabilization methods may alleviate optimization noise in an average sense, they cannot fundamentally address the local geometric mismatch caused by GUI distribution shifts.

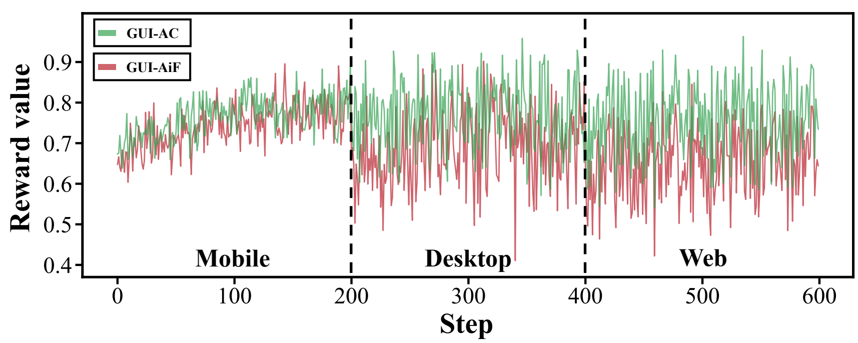

Figure 7: GUI-AC maintains consistently smooth, high-reward trajectories and recovers rapidly after each domain shift, whereas GUI-AiF suffers sharp fluctuations and high variance, highlighting the superior optimization stability of GUI-AC under continual cross-domain learning.

A.3 Reward Dynamics Analysis.

We analyze reward dynamics as a proxy for optimization stability in cross-domain continual learning. Figure 7 reports per-step rewards for GUI-AiF and GUI-AC over the first 200 training steps of each stage. In the initial Mobile stage, both methods yield relatively steady rewards, consistent with stable on-policy improvement when the interface distribution remains largely stationary. After each domain switch, GUI-AiF exhibits sharp reward discontinuities and high-variance oscillations, producing a jagged trajectory that points to unstable rollout outcomes and noisier advantage estimates under distribution shift. In contrast, GUI-AC preserves smooth reward trajectories across all stages: it rapidly re-enters a high-reward regime and sustains a tight reward band with markedly reduced fluctuation amplitude. Taken together, these dynamics indicate that GUI-AC delivers significantly more robust policy optimization in the face of continual cross-domain distribution drift.

A.4 Limitations.

Despite achieving satisfactory performance across all benchmarks, our research has several limitations. First, in extreme reinforcement-learning settings where the base model initially fails on a new interface (e.g., transitioning to an element-dense CAD software), rollout-level certainty may stay near zero for long periods. In this case, both modules become close to constant adjustments, which limits the benefit of certainty-calibrated optimization. Second, due to computational resource constraints, we primarily study a 3B-scale model with three training datasets and three evaluation benchmarks. In future studies, we plan to investigate more complex continual GUI agent scenarios and conduct experiments with larger-scale base models.